

Exercício 3 – Test-Driven Development (TDD)

Nome: Wady Jorge

Curso: Especialização em Tecnologia Java – UTFPR

Data: 15/04/2025

Descrição do Problema

Desenvolver uma calculadora de salário líquido de funcionários com base em seu cargo e salário-base, seguindo regras específicas de desconto. A implementação deve seguir a abordagem de Desenvolvimento Guiado por Testes (TDD), utilizando a linguagem Java e o framework JUnit.

1. Casos de Teste (JUnit)

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class CalculadoraSalarioTest {

    @Test
    public void deveCalcularSalarioDesenvolvedorComDescontoDe20Porcento() {
        Funcionario dev = new Funcionario("João", "joao@email.com", 5000.0,
Cargo.DESENVOLVEDOR);
        CalculadoraSalario calc = new CalculadoraSalario();
        assertEquals(4000.0, calc.calcularSalarioLiquido(dev));
    }

    @Test
    public void deveCalcularSalarioDesenvolvedorComDescontoDe10Porcento() {
        Funcionario dev = new Funcionario("Ana", "ana@email.com", 2500.0,
Cargo.DESENVOLVEDOR);
        CalculadoraSalario calc = new CalculadoraSalario();
        assertEquals(2250.0, calc.calcularSalarioLiquido(dev));
    }

    @Test
    public void deveCalcularSalarioDBAComDescontoDe25Porcento() {
        Funcionario dba = new Funcionario("Carlos", "carlos@email.com", 2500.0, Cargo.DBA);
        CalculadoraSalario calc = new CalculadoraSalario();
        assertEquals(1875.0, calc.calcularSalarioLiquido(dba));
    }
}
```

```

@Test
public void deveCalcularSalarioDBAComDescontoDe15Porcento() {
    Funcionario dba = new Funcionario("Beatriz", "bia@email.com", 1800.0, Cargo.DBA);
    CalculadoraSalario calc = new CalculadoraSalario();
    assertEquals(1530.0, calc.calcularSalarioLiquido(dba));
}

@Test
public void deveCalcularSalarioGerenteComDescontoDe30Porcento() {
    Funcionario gerente = new Funcionario("Lucas", "lucas@email.com", 6000.0,
Cargo.GERENTE);
    CalculadoraSalario calc = new CalculadoraSalario();
    assertEquals(4200.0, calc.calcularSalarioLiquido(gerente));
}

@Test
public void deveCalcularSalarioGerenteComDescontoDe20Porcento() {
    Funcionario gerente = new Funcionario("Julia", "julia@email.com", 3000.0,
Cargo.GERENTE);
    CalculadoraSalario calc = new CalculadoraSalario();
    assertEquals(2400.0, calc.calcularSalarioLiquido(gerente));
}

@Test
public void deveCalcularSalarioTestadorComDescontoDe15Porcento() {
    Funcionario testador = new Funcionario("Tiago", "tiago@email.com", 550.0,
Cargo.TESTADOR);
    CalculadoraSalario calc = new CalculadoraSalario();
    assertEquals(467.5, calc.calcularSalarioLiquido(testador));
}
}

```

2. Classe Funcionario

```

public class Funcionario {
    private String nome;
    private String email;
    private double salarioBase;
    private Cargo cargo;

    public Funcionario(String nome, String email, double salarioBase, Cargo cargo) {
        this.nome = nome;
        this.email = email;
    }
}

```

```

        this.salarioBase = salarioBase;
        this.cargo = cargo;
    }

    public double getSalarioBase() {
        return salarioBase;
    }

    public Cargo getCargo() {
        return cargo;
    }
}

```

3. Enum Cargo

```

public enum Cargo {
    DESENVOLVEDOR,
    DBA,
    TESTADOR,
    GERENTE;
}

```

4. Classe CalculadoraSalario

```

public class CalculadoraSalario {

    public double calcularSalarioLiquido(Funcionario funcionario) {
        double salarioBase = funcionario.getSalarioBase();
        Cargo cargo = funcionario.getCargo();

        switch (cargo) {
            case DESENVOLVEDOR:
                return salarioBase >= 3000 ? salarioBase * 0.8 : salarioBase * 0.9;
            case DBA:
            case TESTADOR:
                return salarioBase >= 2000 ? salarioBase * 0.75 : salarioBase * 0.85;
            case GERENTE:
                return salarioBase >= 5000 ? salarioBase * 0.7 : salarioBase * 0.8;
            default:
                throw new IllegalArgumentException("Cargo não reconhecido.");
        }
    }
}

```